Applied Machine Learning

Dr. Sunil Kumar Telagamsetti
Department of Electrical Engineering
Email: Sunilkumar.Telagam.Setti@hig.se
Sunil kumar Telagamsetti | LinkedIn

Supervised Machine Learning

Content

- Linear regression
- Logistic regression
- Assignment 1
- Practical by Nusyba

- Assume you are planning to buy a smartphone? What are the things that comes into your mind?

  - Memory
  - Cam specifications
  - RAM

  - Features?

  - Price of your smart phone depend on features, some features may impact your significantly
  - Price of house depends on features like area, number of rooms, recently renovated, pool etc

"All models are wrong, but some are useful"

George E. P. Box

All models are simplifications of reality and never perfectly accurate

Supervised Learning

- Algorithm learns to map input data to known output data
- Given
  - A set of input features X (input variables, independent variables,)
  - A target value Y
  - A set of training examples where the values for the input features and the target features are given for each example
  - A new example, where only the values for the input features are given
- Predict the values for the target features for the new example
  - Classification when Y is discrete
  - Regression when Y is continuous

Note:

- Input Vector: The collection of features for a single instance.
- Target/Label: The outcome or output the model is trying to predict.
- While features are the individual attributes, the entire set of data passed to the model to produce a prediction is often referred to simply as the **input** or **data**

Challenges involved in Supervised Learning

Insufficient or biased data: Lack of data or biased data can lead to poor model performance or inaccurate predictions

Overfitting: When the model is too complex, it can fit the training data too closely, leading to poor generalization performance on new data.

Feature engineering: choosing and engineering the right set of features can be a time-consuming and subjective process that requires domain expertise.

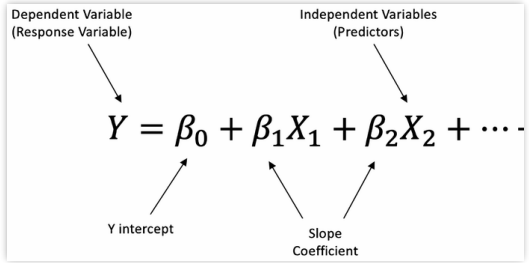
Model Selection: Choosing the right model and hyperparameters for a given problem can be challenging and require extensive experimentation and tuning.

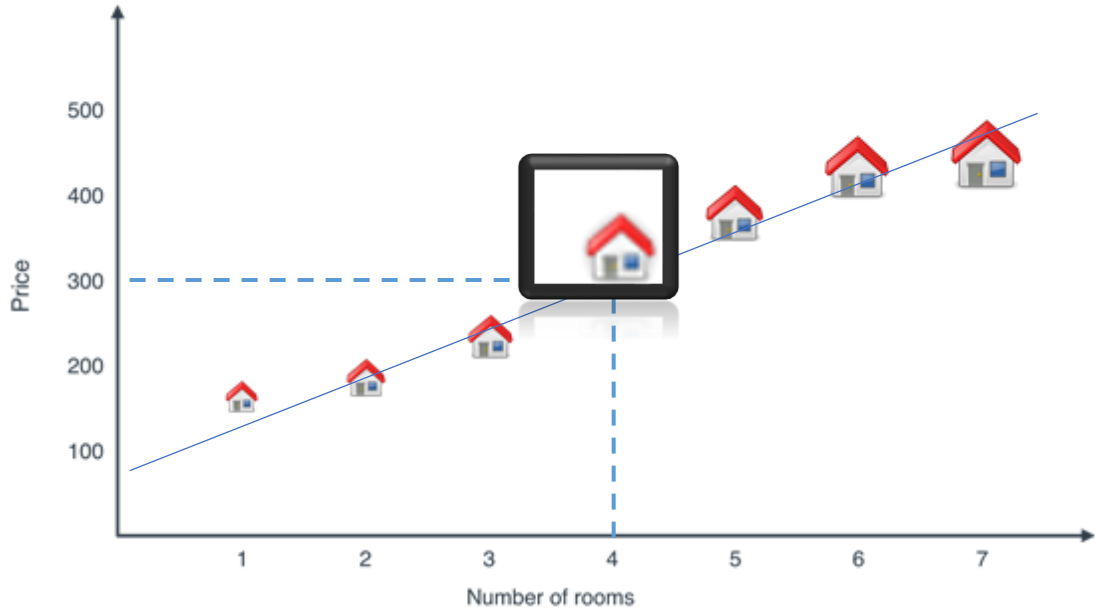
Interpretability: Understanding why a model makes certain predictions or decisions can be difficult, especially for complex models like deep neural networks.

Linear Regression

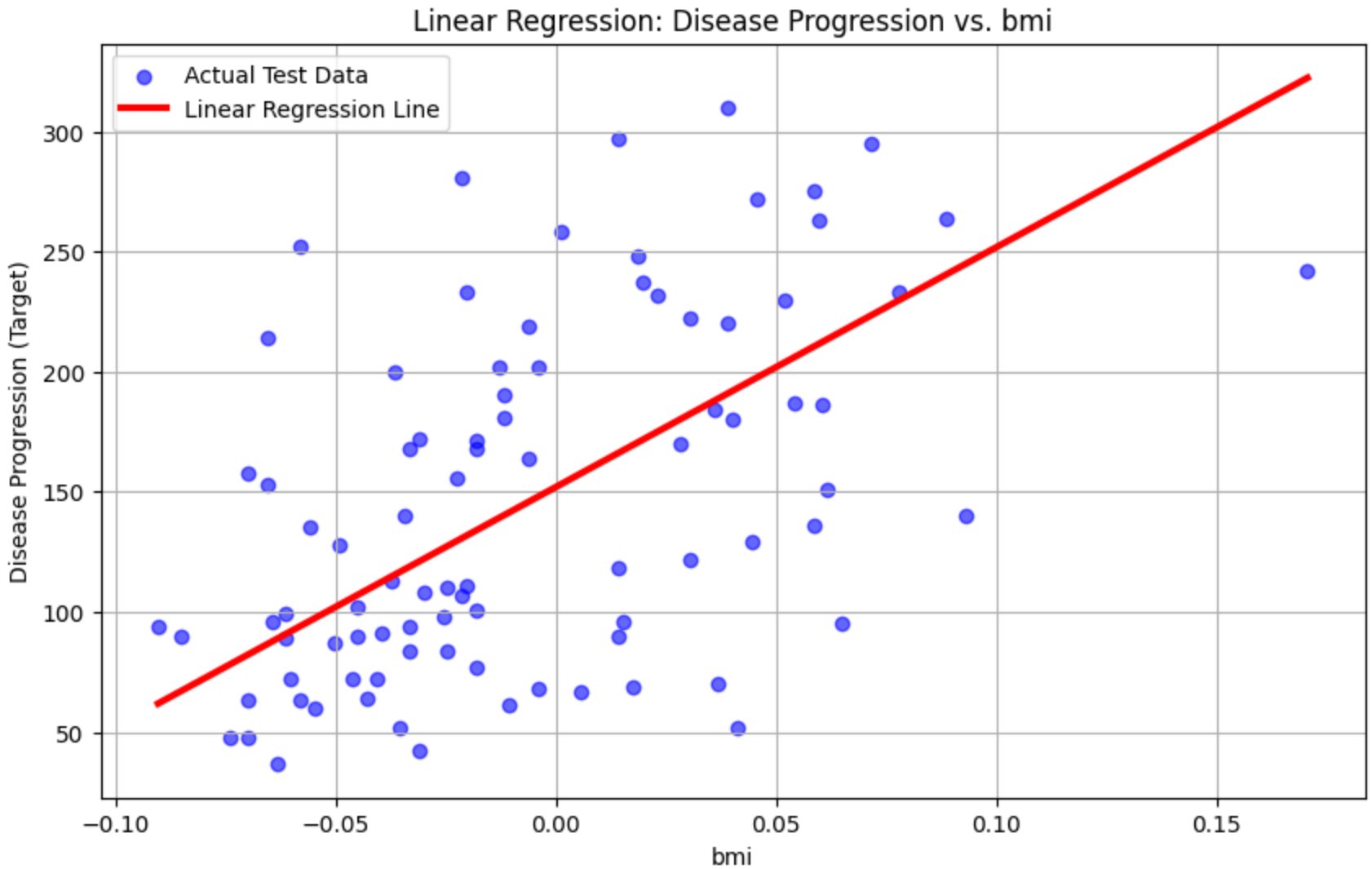
Classification predicts categorical classes (like anomaly), while regression predicts continuous numerical values (like age, income, or temperature)

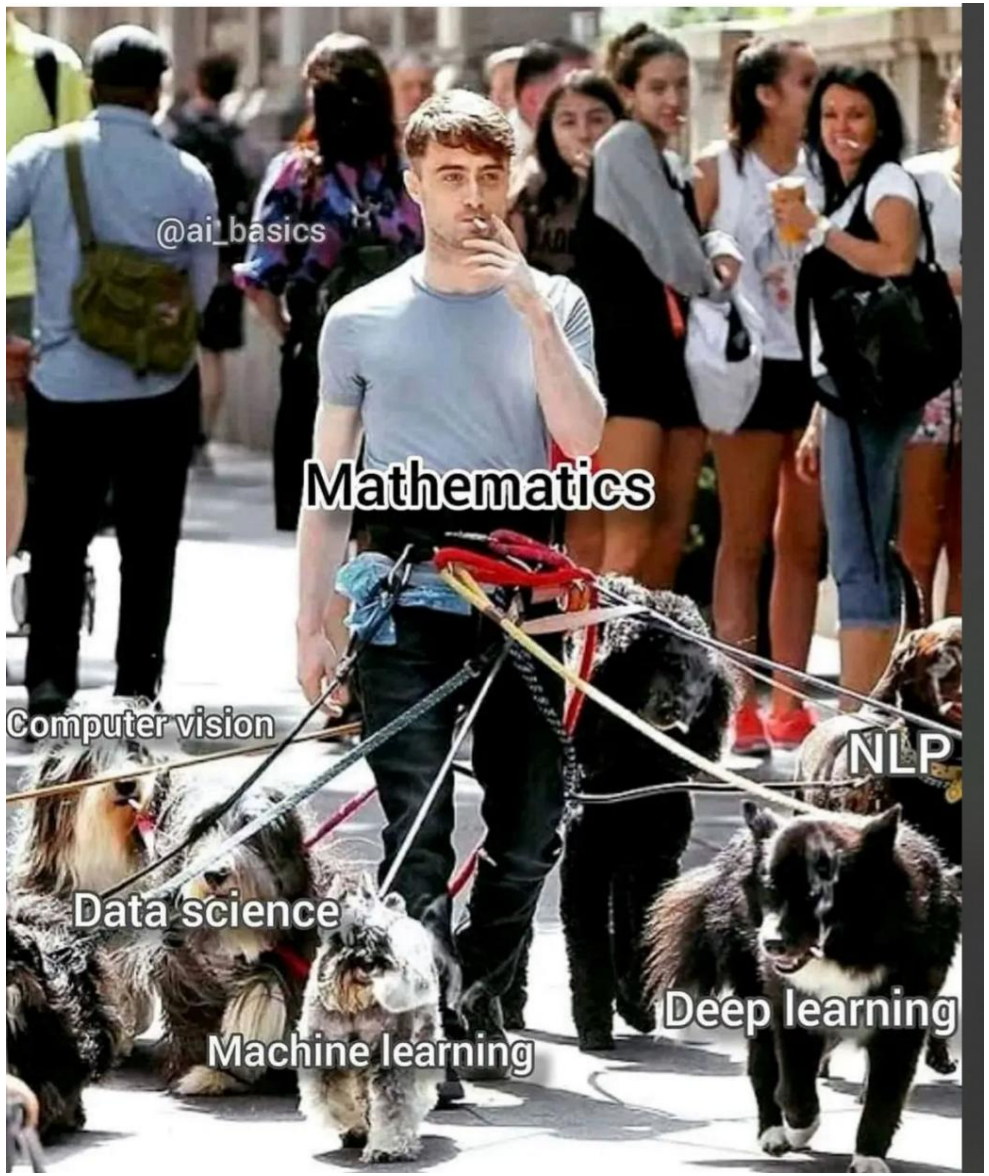
Rooms=4. Price?



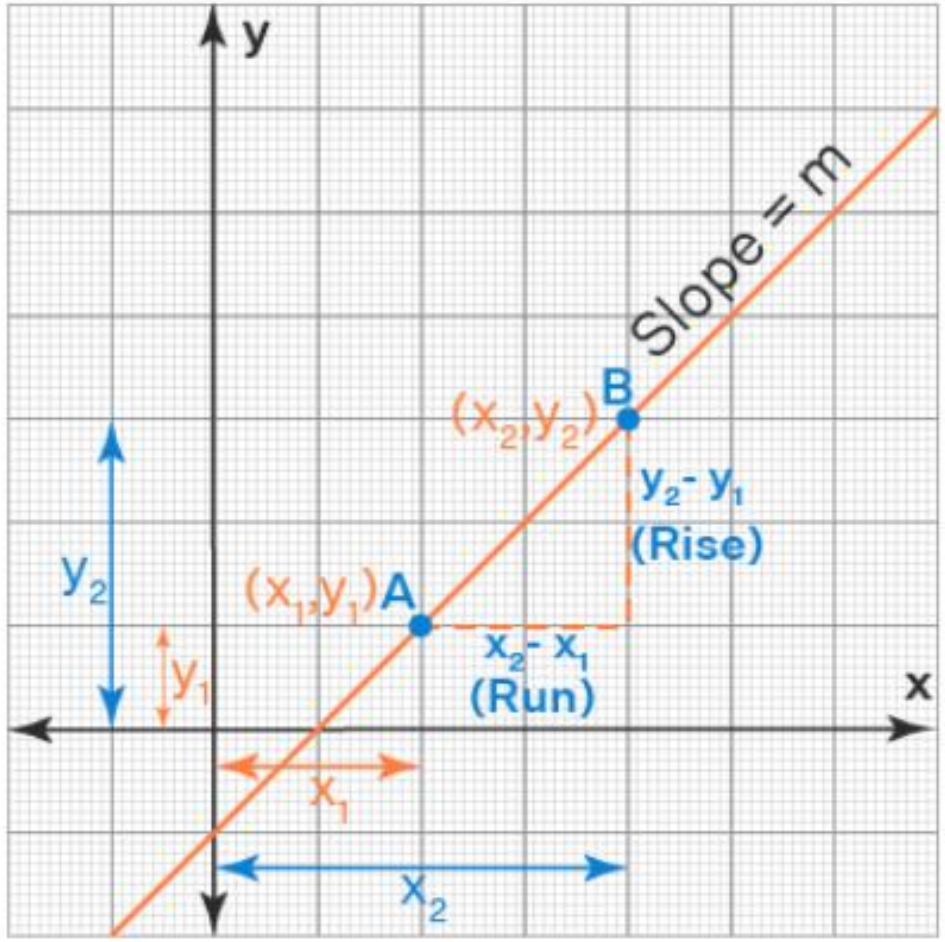


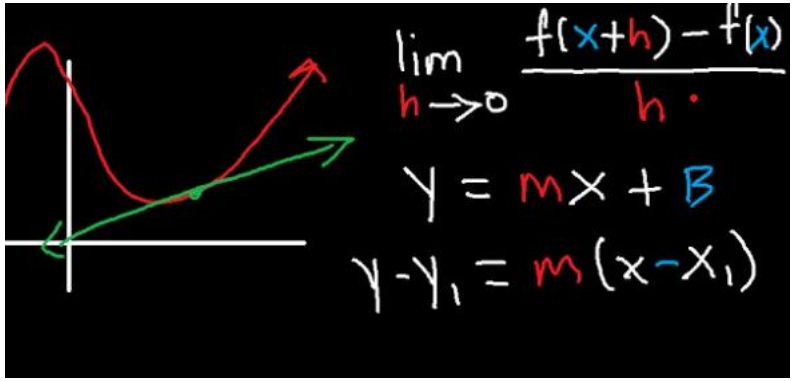
y=100+0.5(x)


Linear Regression: Disease Progression vs. bmi
Actual Test Data
Linear Regression Line
Disease Progression (Target)
300
250
200
150
100
50
−0.10
−0.05
0.00
0.05
0.10
0.15
bmi

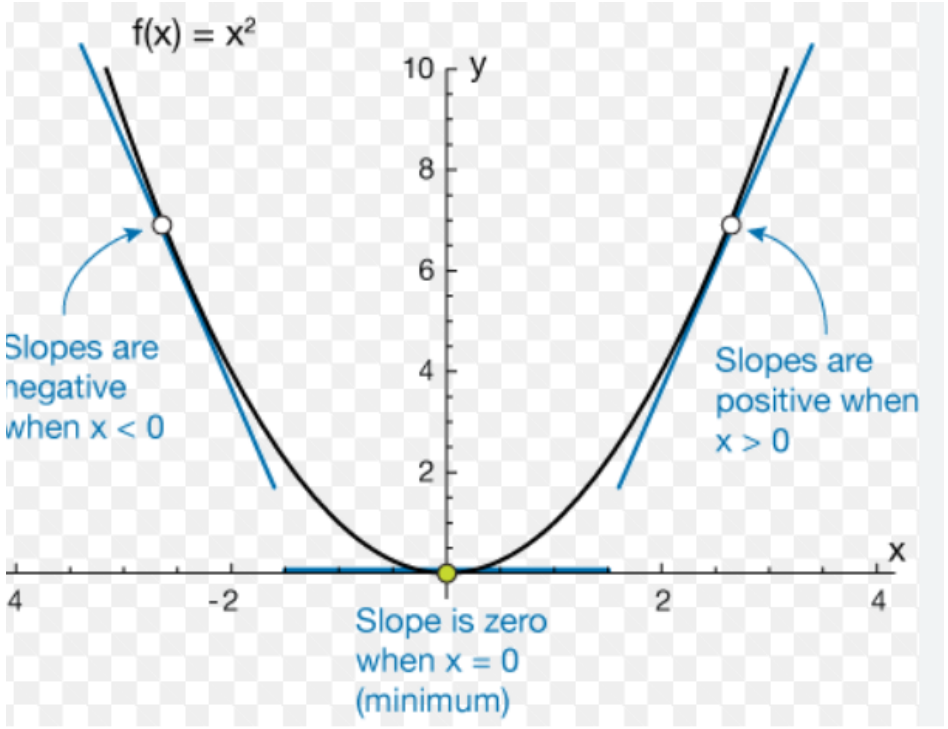

@ai_basics
Mathematics
Computer vision
NLP
Data science
Deep learning
Machine learning

Review of Mathematics





Slope $= \frac{y_2 - y_1}{x_2 - x_1}$



- Derivative gives slope
- Positive slope-increasing
- Negative slope-decreasing
- Slope=zero, max or min

- Matrix multiplication

A= $\theta_0 + \theta_1 x^{(i)}$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} \qquad x^{(i)} = \begin{bmatrix} 1 \\ x^{(i)} \end{bmatrix}$$

$$X = \begin{bmatrix} 1 & x_1^{(1)} \\ 1 & x_1^{(2)} \\ \vdots & \vdots \\ 1 & x_1^{(m)} \end{bmatrix}, \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} \qquad \hat{y} = X\theta \qquad \hat{y} = \begin{bmatrix} \theta_0 + \theta_1 x_1^{(1)} \\ \theta_0 + \theta_1 x_1^{(2)} \\ \vdots \\ \theta_0 + \theta_1 x_1^{(m)} \end{bmatrix}$$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$h(x) = \sum_{i=0}^{n} \theta_i x_i = \theta^T x,$$

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \cdots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \cdots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & \cdots & x_n^{(m)} \end{bmatrix}$$

$$\hat{y} = \begin{bmatrix} \theta_0 + \theta_1 x_1^{(1)} + \cdots + \theta_n x_n^{(1)} \\ \theta_0 + \theta_1 x_1^{(2)} + \cdots + \theta_n x_n^{(2)} \\ \vdots \\ \theta_0 + \theta_1 x_1^{(m)} + \cdots + \theta_n x_n^{(m)} \end{bmatrix}$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix}$$

$$\hat{y}^{(i)} = \theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_n x_n^{(i)}$$

Binary decisions:

Assume, Probability of a given person having tumor =P

Probability of a given person having tumor =1-P

- Derivative
- Difference
- Matrix multiplication
- Sigmoid
- Binary classification

$$g(z) = \frac{1}{1+e^{-z}}$$

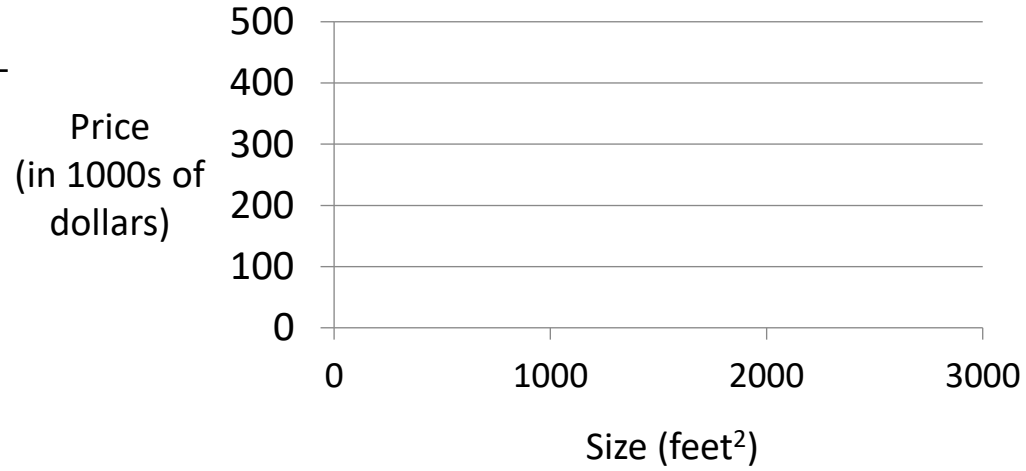
$$\begin{aligned}
\frac{d}{dz}g(z) &= \frac{d}{dz}\frac{1}{1+e^{-z}} \\
&= -\frac{1}{(1+e^{-z})^2}\frac{d}{dz}\left(1+e^{-z}\right) \\
&= -\frac{1}{(1+e^{-z})^2}\left(-e^{-z}\right) \\
&= \frac{1+e^{-z}-1}{(1+e^{-z})^2} \\
&= \frac{1}{1+e^{-z}}\left(1-\frac{1}{1+e^{-z}}\right) \\
&= g(z)[1-g(z)]
\end{aligned}$$

Linear regression with one variable

Example: Housing Prices

Training set of housing prices

Size in feet^2 (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
…	…



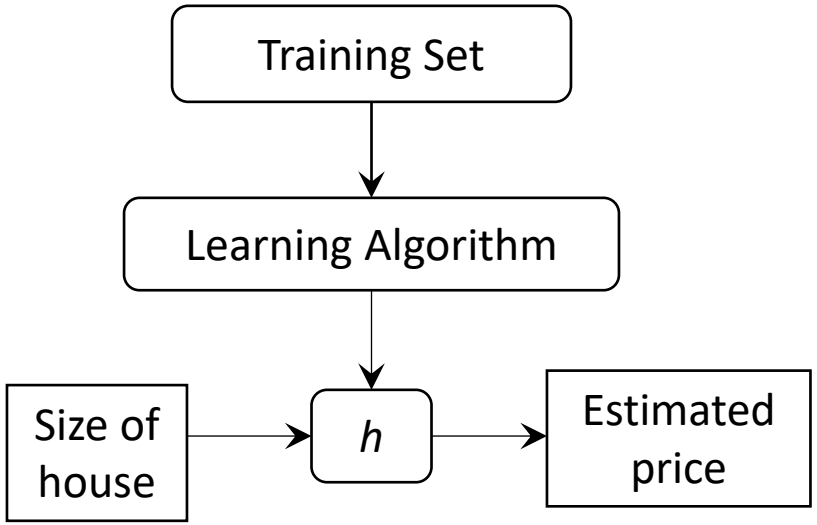
Supervised Learning

Given the "right answer" for each example in the data.

Regression Problem

Predict real-valued output

Linear regression



How do we represent *h* ?

Linear regression with one variable.
Univariate linear regression.

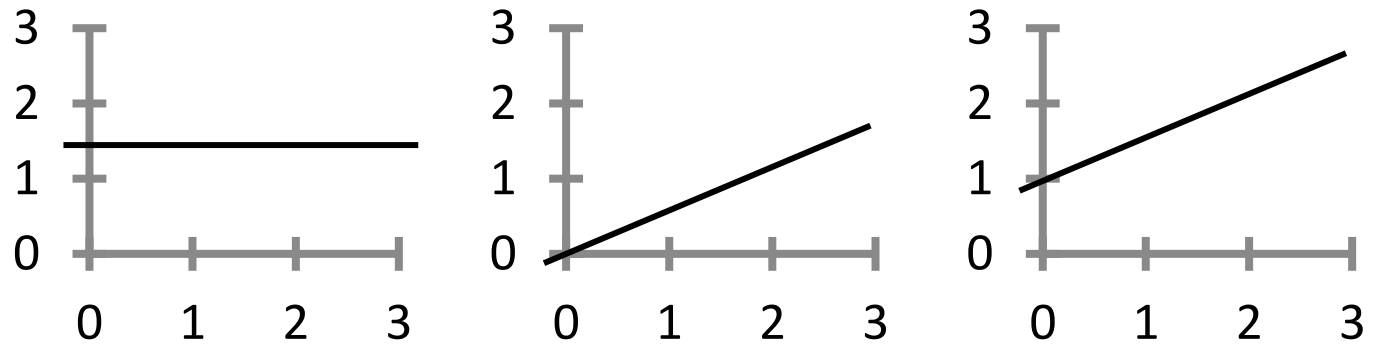
Hypothesis:

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Linear regression

Our goal is to find the coefficients

$$h_\theta(x) = \theta_0 + \theta_1 x$$



$\theta_0 = 1.5$
$\theta_1 = 0$

$\theta_0 = 0$
$\theta_1 = 0.5$

$\theta_0 = 1$
$\theta_1 = 0.5$

Linear regression

Idea: Choose θ_0, θ_1 so that $h_\theta(x)$is close to y for our training examples (x, y)

Notation:

m = Number of training examples

x's = "input" variable / features

y's = "output" variable / "target" variable

θ_i's: Parameters

How to choose θ_i's ?

Linear regression

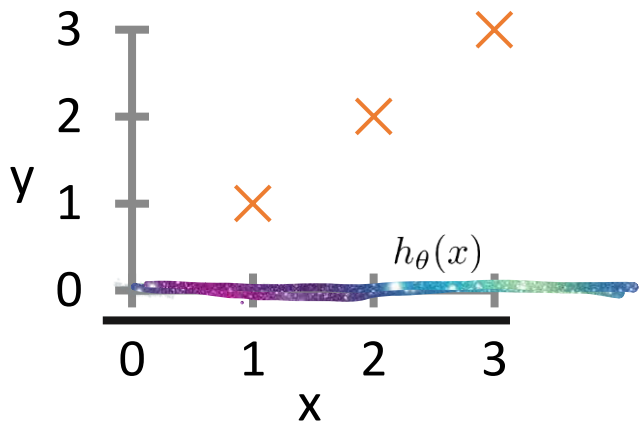
	Simplified $\theta_0 = 0$
Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$	$h_\theta(x) = \theta_1 x$
Parameters: θ_0, θ_1	θ_1
Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$	$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$
Goal: $\underset{\theta_0,\theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$	$\underset{\theta_1}{\text{minimize}}\ J(\theta_1)$

$\theta_1 = 0$

Assume that y=x is the best fit

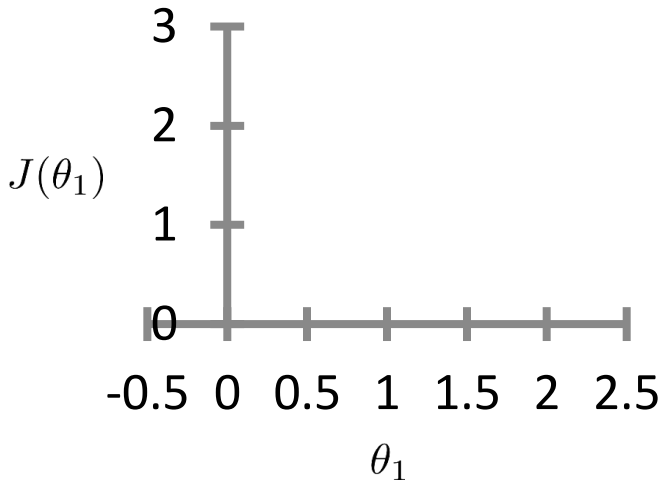
$h_\theta(x)$

(for fixed θ_1, this is a function of x)



$J(\theta_1)$

(function of the parameter θ_1)

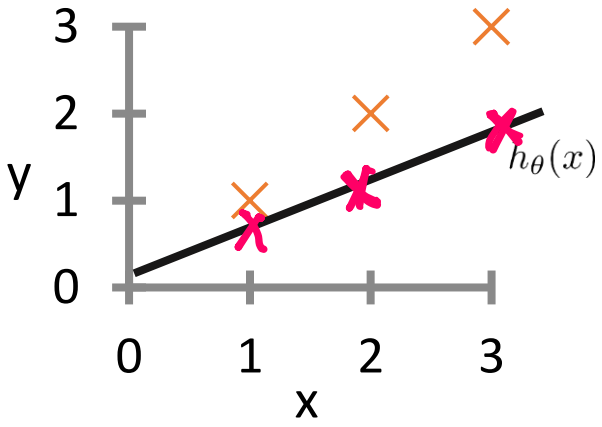


$\theta_1 = 0.5$

$h_\theta(x) = 0.5x$

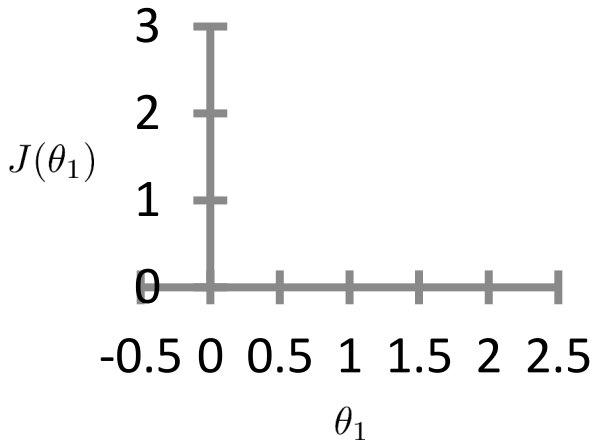
$h_\theta(x)$

(for fixed θ_1, this is a function of x)



$J(\theta_1)$

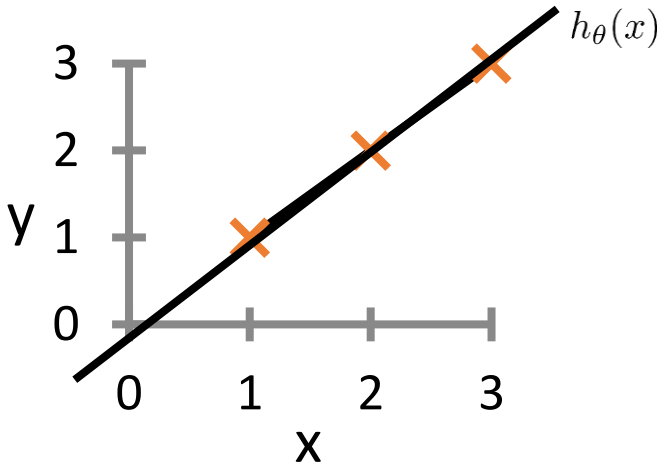
(function of the parameter θ_1)



$\theta_1 = 1$

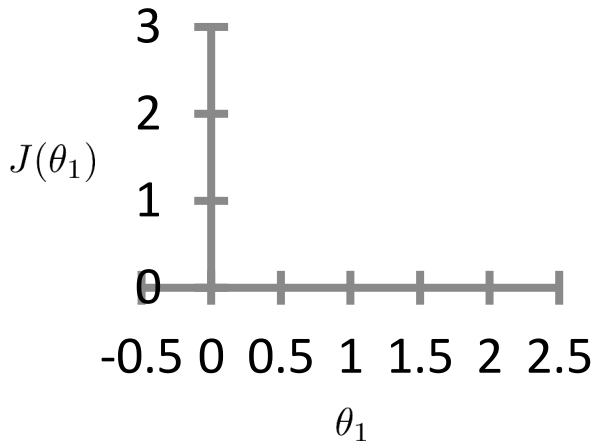
$h_\theta(x)$

(for fixed θ_1, this is a function of x)

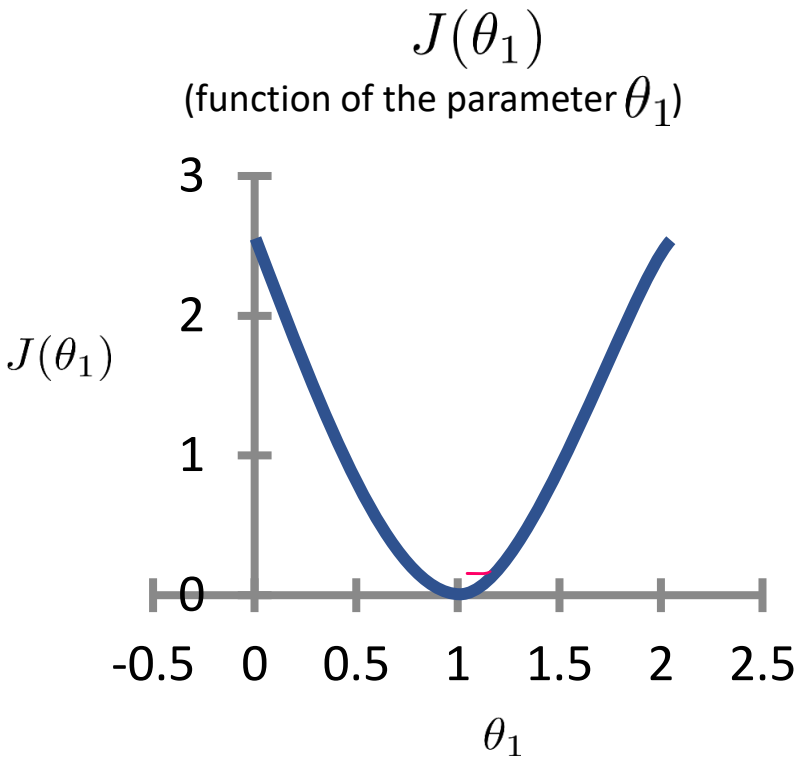


$J(\theta_1)$

(function of the parameter θ_1)



Cost function



Linear regression

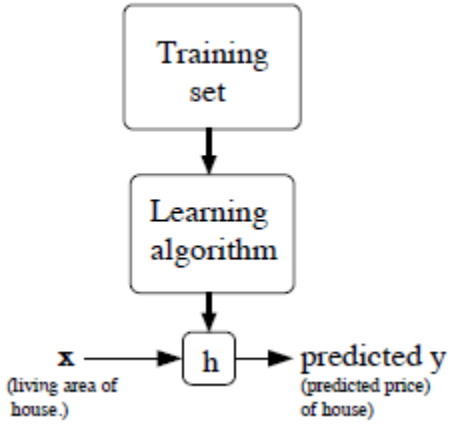
Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

Linear regression



$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$h(x) = \sum_{i=0}^{n} \theta_i x_i = \theta^T x,$$

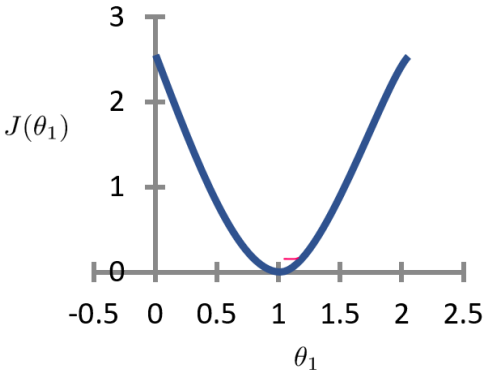
We define the **cost function**:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2.$$

Tunable parameters (often just called "parameters") and hyperparameters are distinct variables in machine learning. Parameters are learned automatically from data during training (e.g., weights, coefficients in regression), while hyperparameters are set manually before training to guide the learning process

LMS algorithm

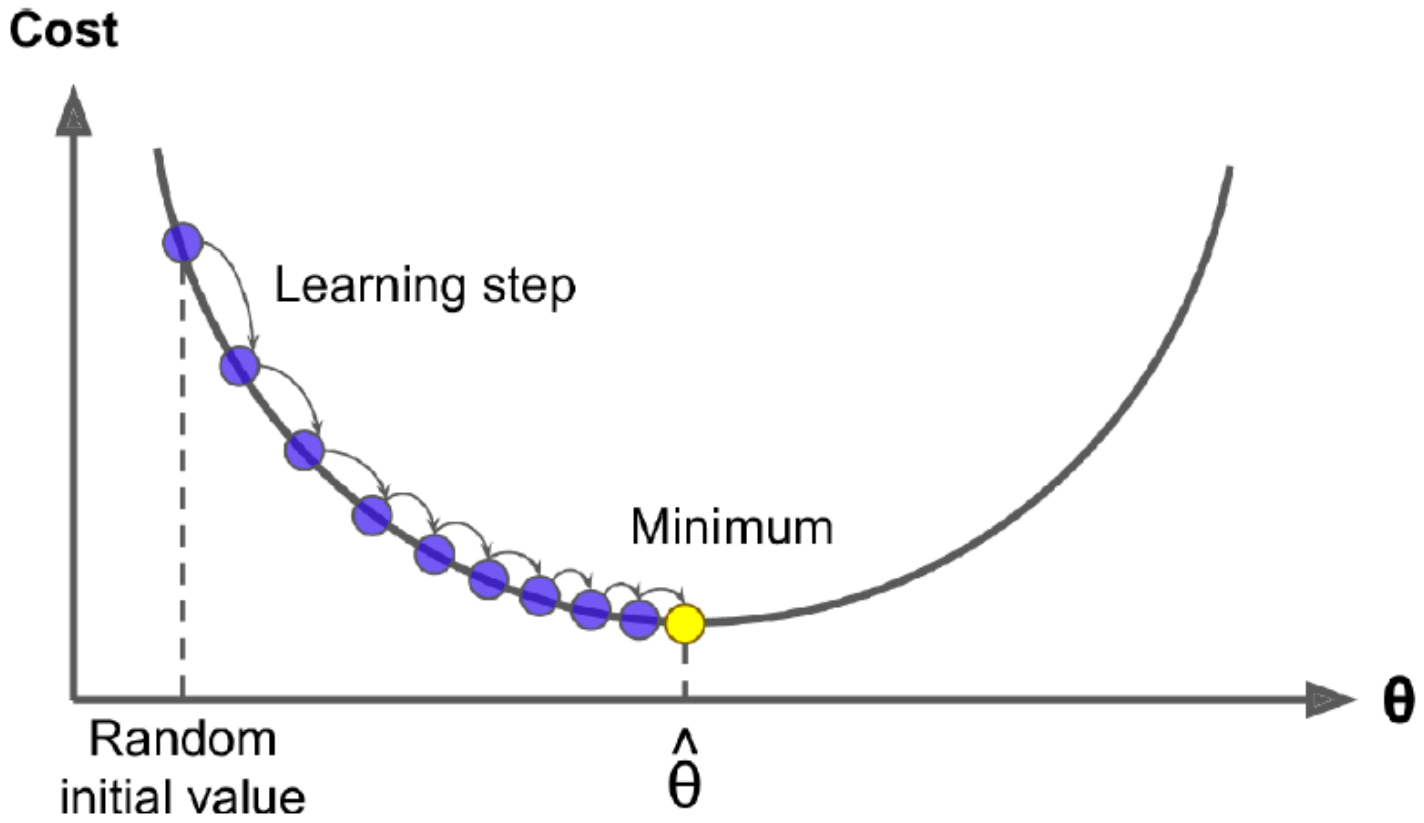
$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta).$$



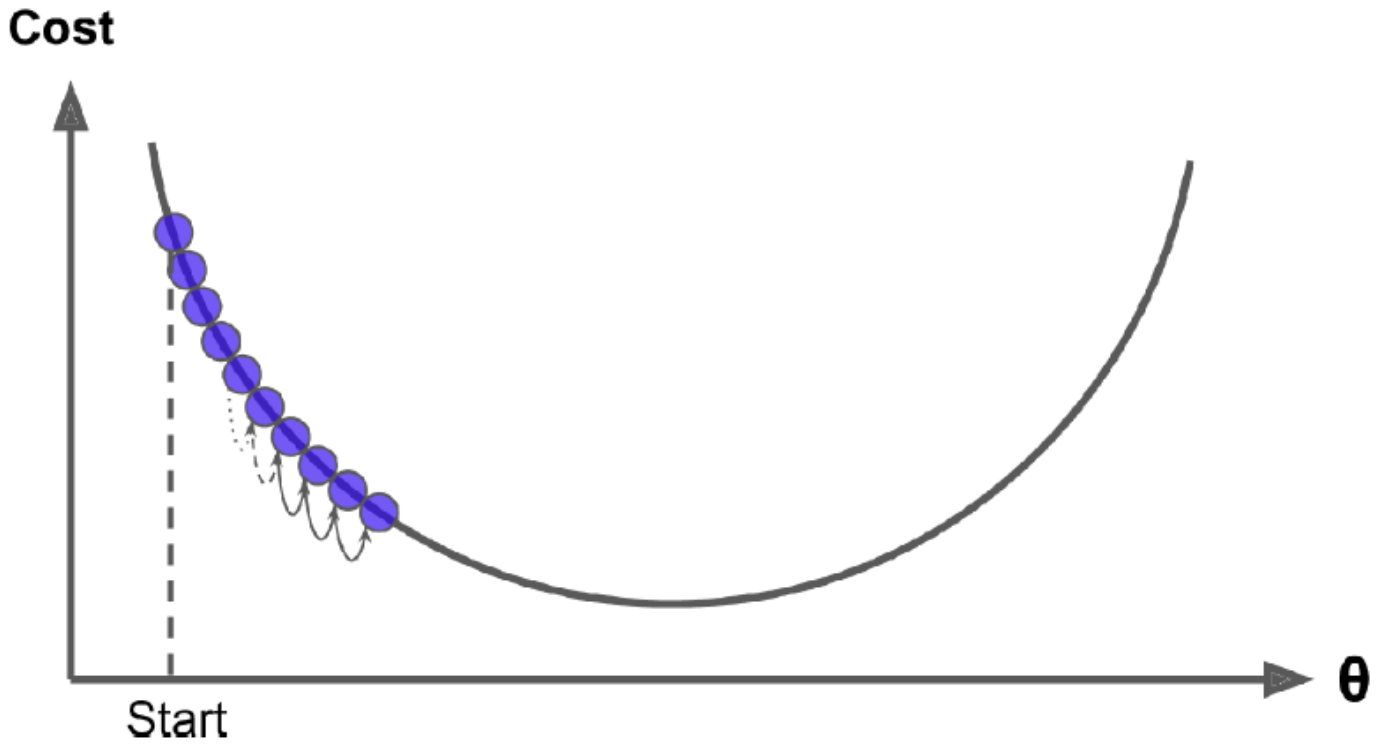
$$\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_\theta(x) - y)^2 \\
&= 2 \cdot \frac{1}{2} (h_\theta(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_\theta(x) - y) \\
&= (h_\theta(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^{n} \theta_i x_i - y \right) \\
&= (h_\theta(x) - y)\, x_j
\end{aligned}$$

$$\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)}) \right) x_j^{(i)}.$$

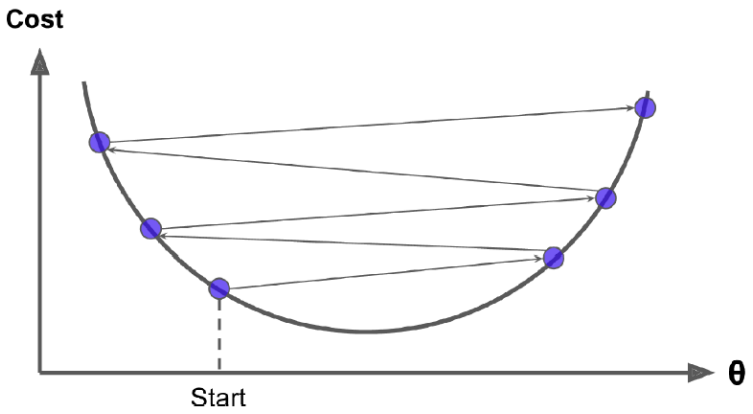
Learning Rate



If α is too small, gradient descent can be slow.



If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.



Feature scaling ensures all input features have a similar range of values, preventing variables with larger magnitudes from dominating the model. It enhances performance for distance-based algorithms, speeds up gradient descent convergence, and ensures equal contribution from all features

Min-Max Scaling

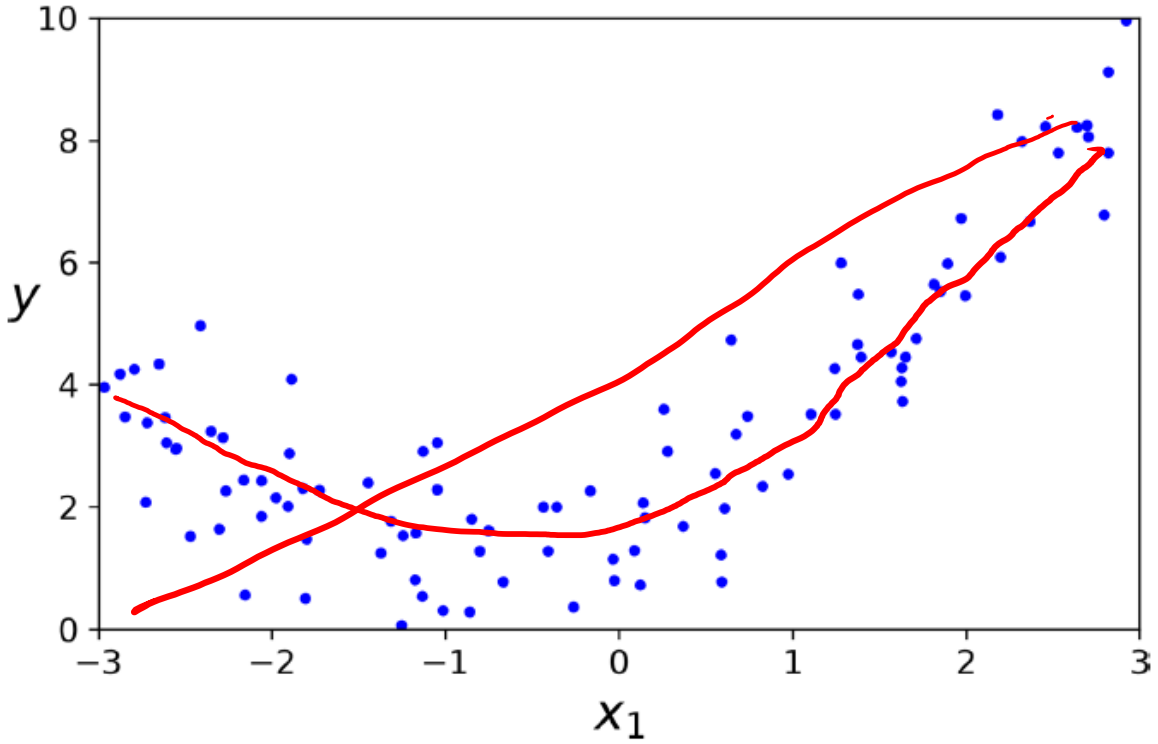
Min-Max scaling transforms data into a fixed range, usually 0 to 1.

Formula:

$$x' = \frac{x - \min}{\max - \min}$$

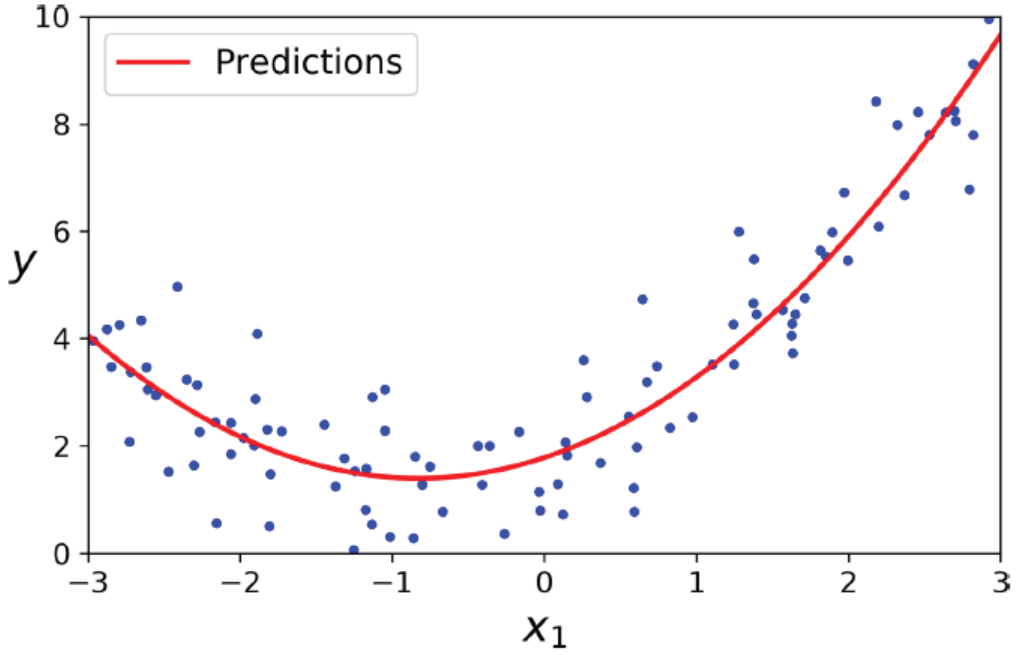
Min-Max Scaling Formula

Polynomial Regression



Generated nonlinear and noisy dataset

Polynomial Regression



Polynomial Regression model predictions

Locally Weighted Regression

- Fit θ to minimize CF:

$$\bullet\ J(\theta) = \sum_{i=1}^{m} w^{(i)} (h_{\theta} (x^{(i)}) - y^{(i)})^2$$

- $w^{(i)}$ is the weight function

$$\bullet\ w^{(i)} = \exp[-\frac{(x^{(i)} - x)^2}{2\tau^2}]$$

- X-> Test feature for which we want to make predictions
- $x^{(i)}$-> feature for i^{th} training example
- $w^{(i)}$ -> lies between 0 and 1. It tells us how much weightage should be given to the i^{th} training feature.
- τ -> bandwith paramter ; decides how many nearby training examples to be considered while fitting the line

•**Distance Metric:** The weights are typically calculated using a Gaussian-like kernel function that measures the distance between the training examples and the current point, controlled by a bandwidth parameter
that defines the size of the neighborhood.

Regression to classification

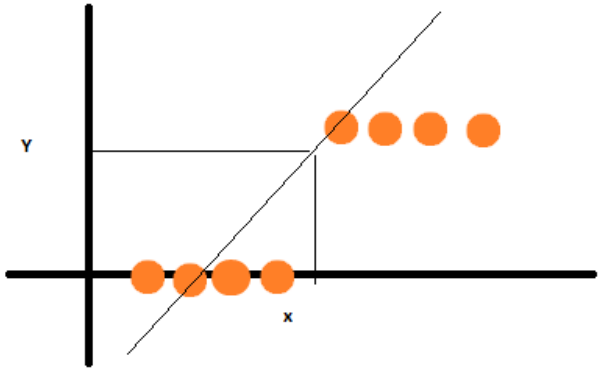
- Logistic regression: Although name suggests regression it is classification

Why linear regression fails?

- Suppose we want to classify

Y= {0,1} and **X** are **data samples**. It is binary classification

- Try it with linear Regression



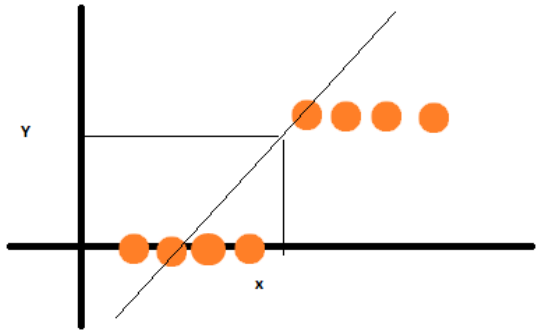
- It is really a good fit but what happens if I add a new data to given data set

Why linear regression fails?

- Suppose we want to classify

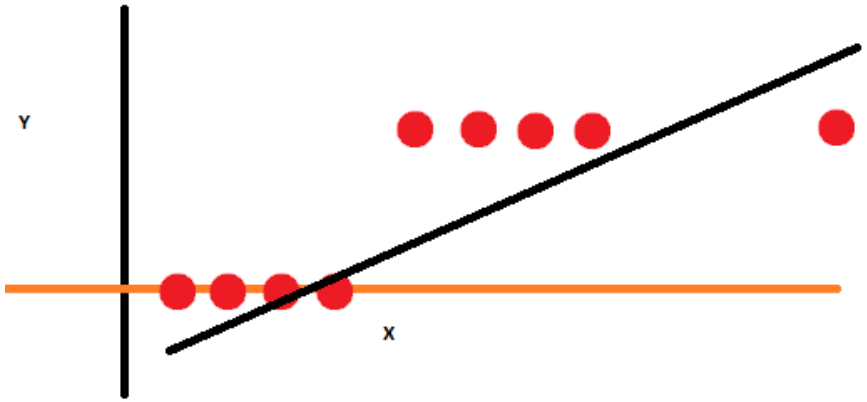
Y= {0,1} and **X** are **data samples**. It is binary classification

- Try it with linear Regression



- It is really a good fit but what happens if we add a new data to given data set

- It's really a bad solution. Linear Regression didn't worked



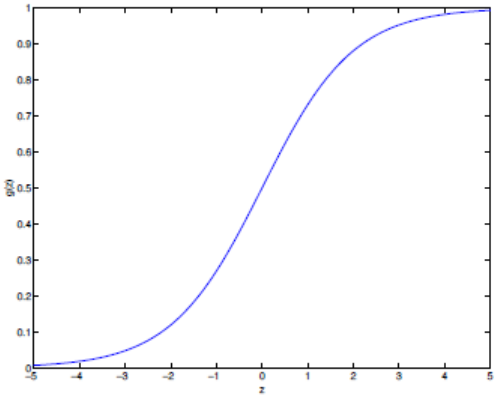
If there is problem, there is solution and the solution is **Logistic Regression.**

Linear regression to Logistic regression

$$h(x) = \sum_{i=0}^{n} \theta_i x_i = \theta^T x,$$

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}},$$

$$g(z) = \frac{1}{1 + e^{-z}}$$



assume that

$$\begin{aligned} P(y = 1 \mid x; \theta) &= h_\theta(x) \\ P(y = 0 \mid x; \theta) &= 1 - h_\theta(x) \end{aligned}$$

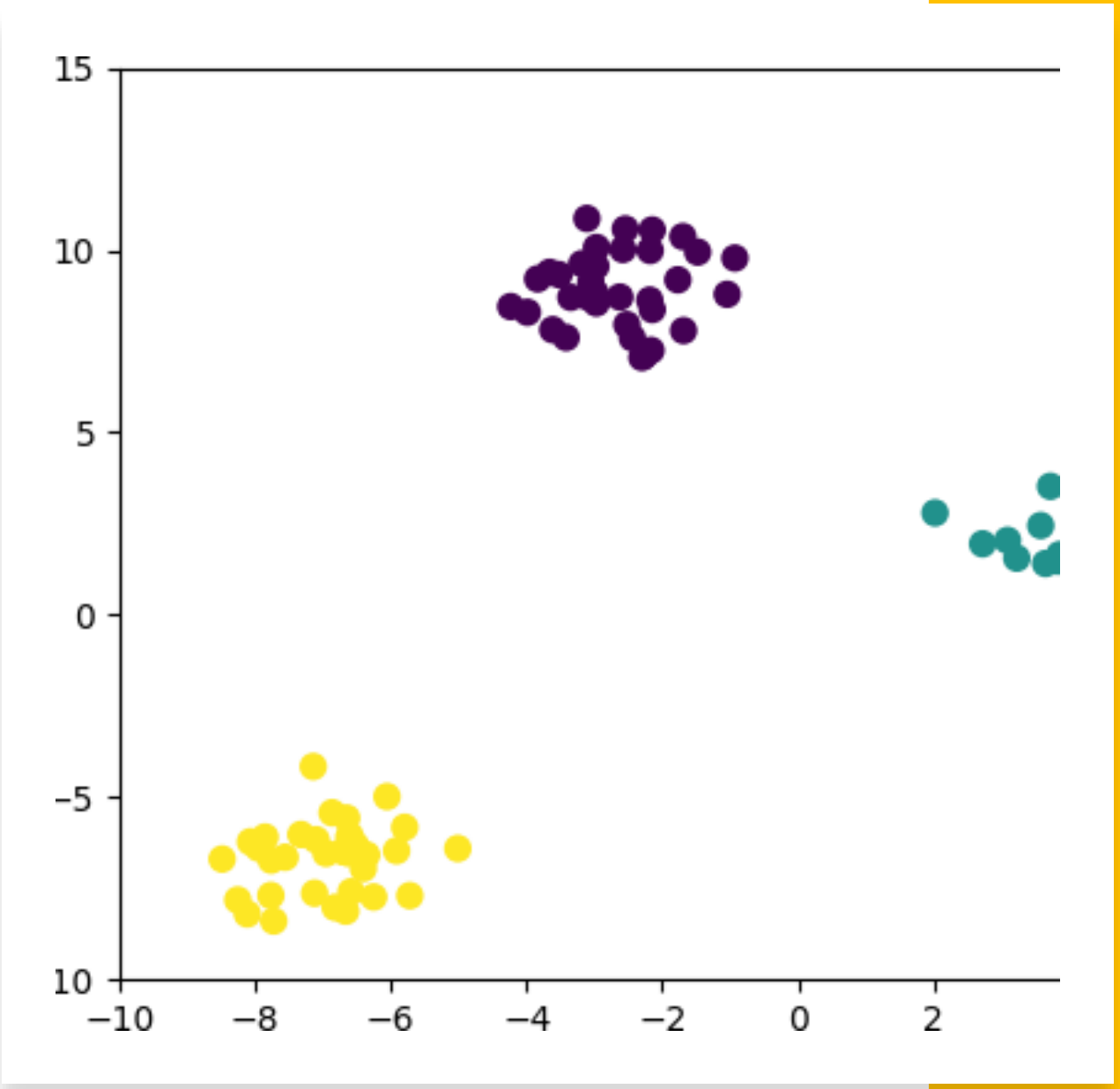
$$p(y \mid x; \theta) = (h_\theta(x))^y \, (1 - h_\theta(x))^{1-y}$$

$$\begin{aligned} \ell(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^{m} y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)})) \end{aligned}$$

$$\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)}) \right) x_j^{(i)}$$

Multiclass Classification

- Many classification models are binary models. We will also need classification models for multiclass problem.

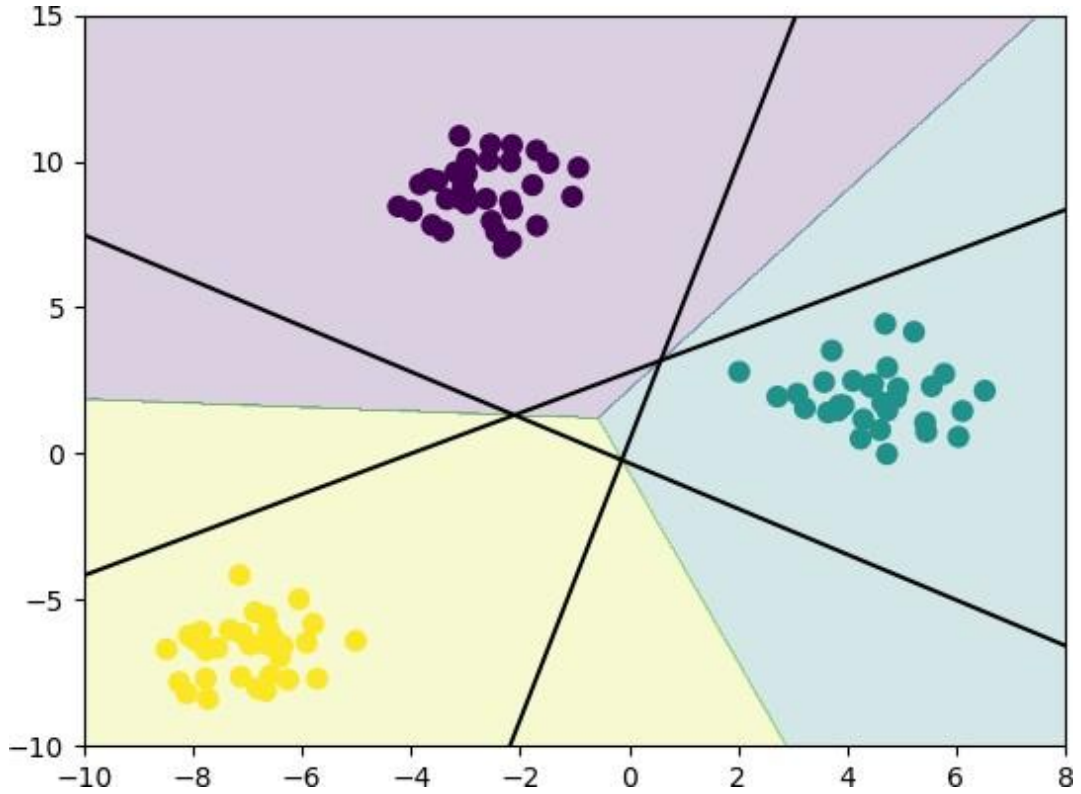


One vs all or One vs rest

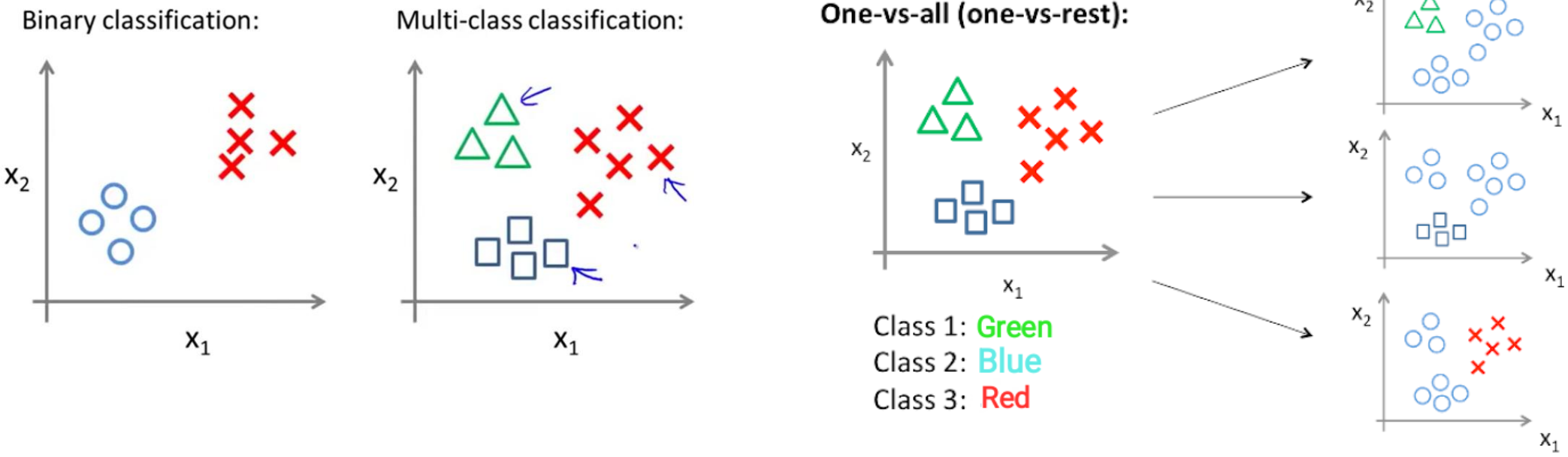
A common technique to extend a binary classification algorithm to multiclass classification is **one-vs-rest** approach.
Let say there are 3 classes: class A, B and C.

- a binary model is built for class A against the rest of the classes (ie B and C).
- similarly build another 2 binary models for class B and class C.

- to make a prediction on a test point, the score of the 3 binary models are calculated. The highest score of the 3 binary model is identified and the corresponding class is returned as prediction.



Multiclass classification



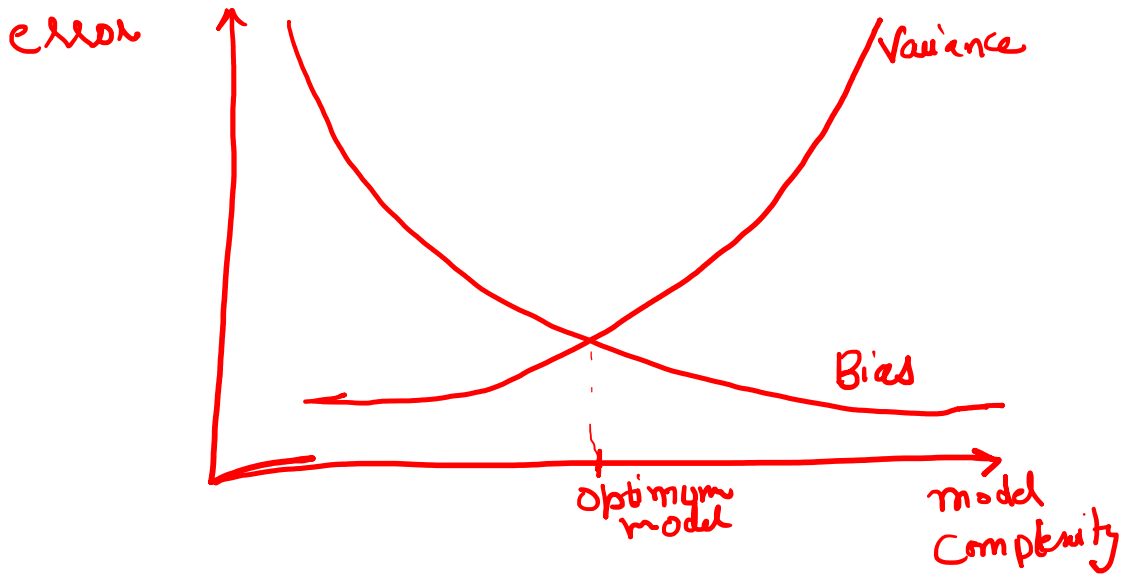
Bias versus Variance Trade off

- *Bias:* This part is due to wrong assumptions, such as assuming that the data is linear when it is actually quadratic. A high-bias model is most likely to underfit the training data.

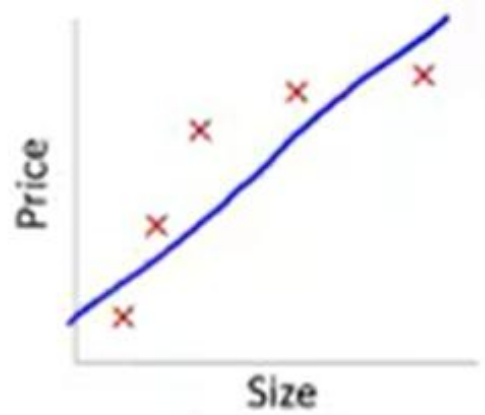
- *Variance:* This part is due to the model's excessive sensitivity to small variations in the training data. A model with many degrees of freedom (such as a high-degree polynomial model) is likely to have high variance and thus overfit the training data.

- *Note: Increasing a model's complexity will typically increase its variance and reduce its bias. Conversely, reducing a model's complexity increases its bias and reduces its variance. This is why it is called a trade-off.*

Bias versus Variance Trade off

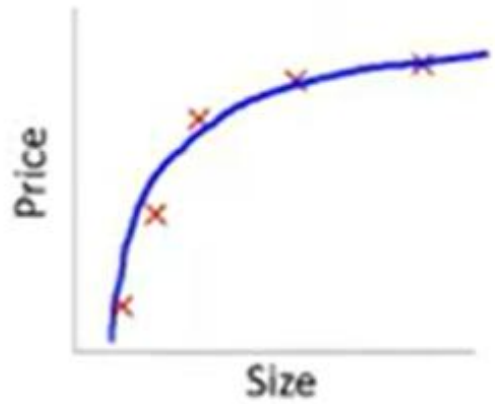


low bias & low Variance ⇒ Good Model



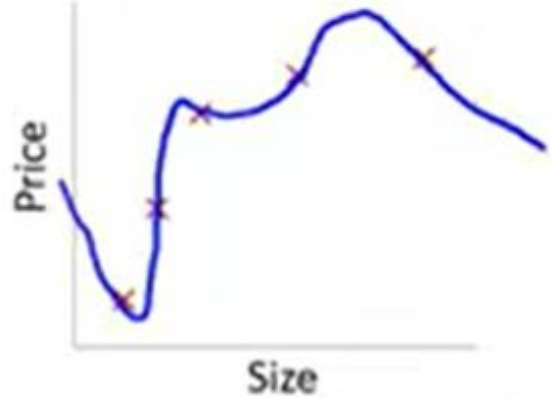
$\rightarrow \theta_0 + \theta_1 x$

"Underfit" "High bias"



$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2$

"Just right"

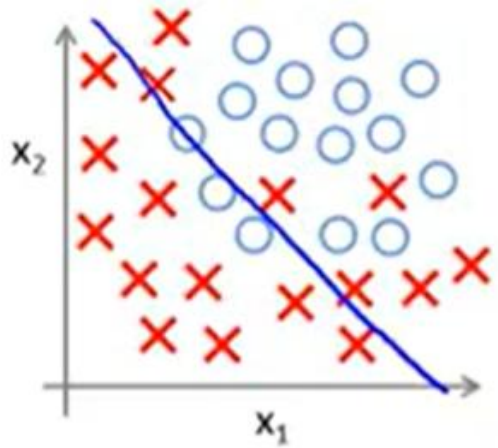


$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

"Overfit" "High variance"

Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

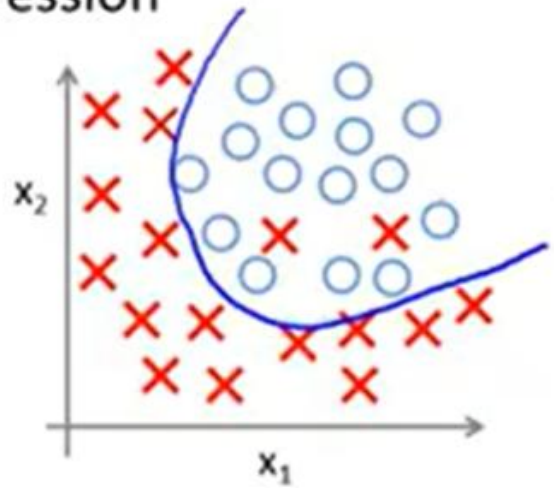
Example: Logistic regression



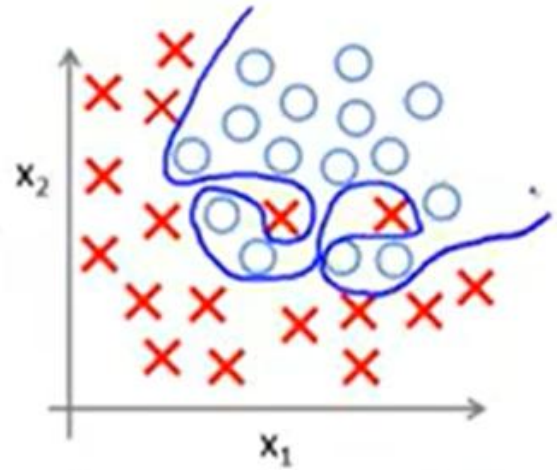
$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$

(g = sigmoid function)

"Underfit"



$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$



$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$

Addressing overfitting:

Reduce number of features.

— Manually select which features to keep.

Regularization.

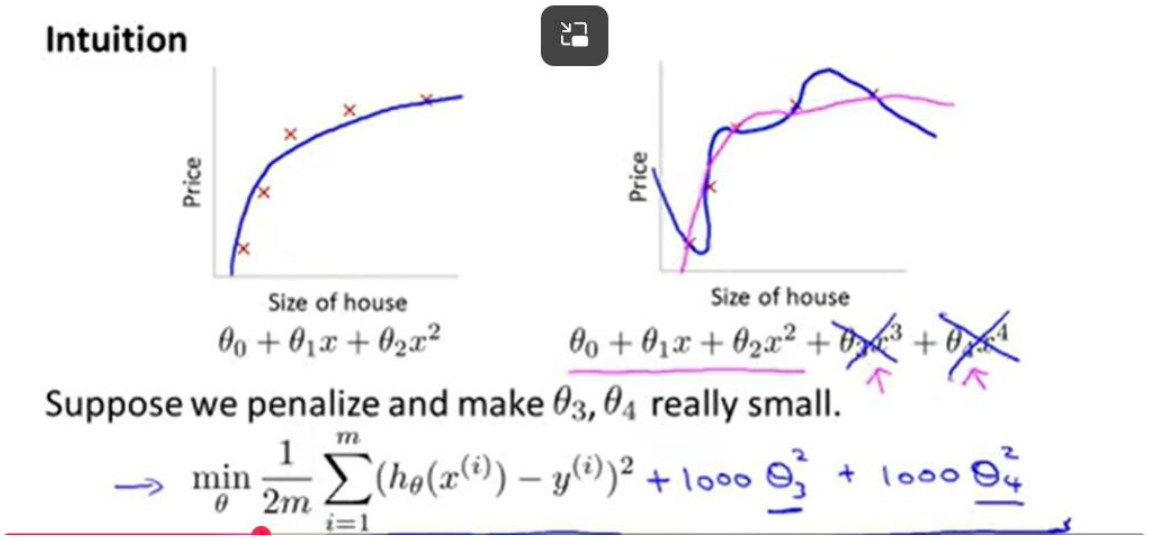
— Keep all the features, but reduce magnitude/values of parameters θ_j.

— Works well when we have a lot of features, each of which contributes a bit to predicting y.

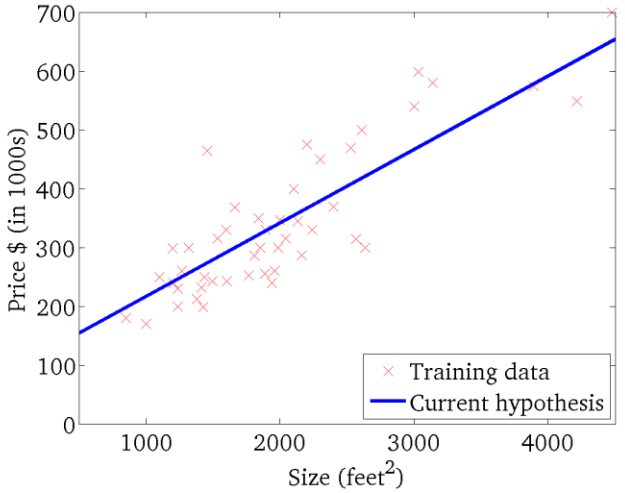
Regularized Linear Models

Regularized Linear Models

- A simple way to regularize a polynomial model is to reduce the number of polynomial degrees.
- For a linear model, regularization is typically achieved by constraining the weights of the model. We will now look at Ridge Regression, Lasso Regression, and Elastic Net, which implement three different ways to constrain the weights.



Ridge Regression



$$\text{CF} = \frac{1}{m}\sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2 + \lambda \times (\text{slope})^2$$

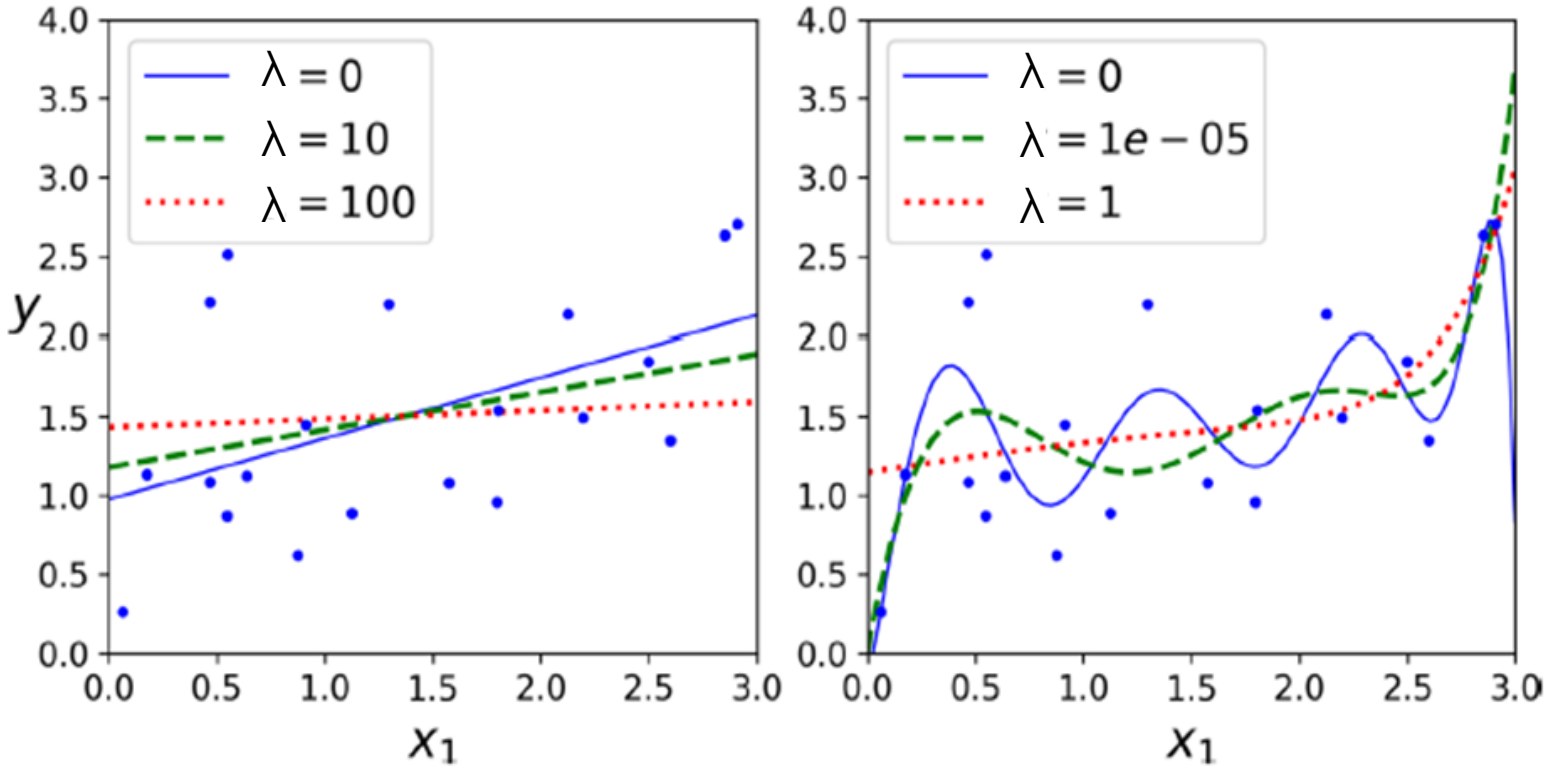
$$\text{Slope} = \sum_{j=1}^{n} (\theta_j)^2$$

λ –> determines how severe that penalty should be

λ => 0 to ∞

Larger the λ, smaller will be the slope of ridge regression line

Ridge Regression

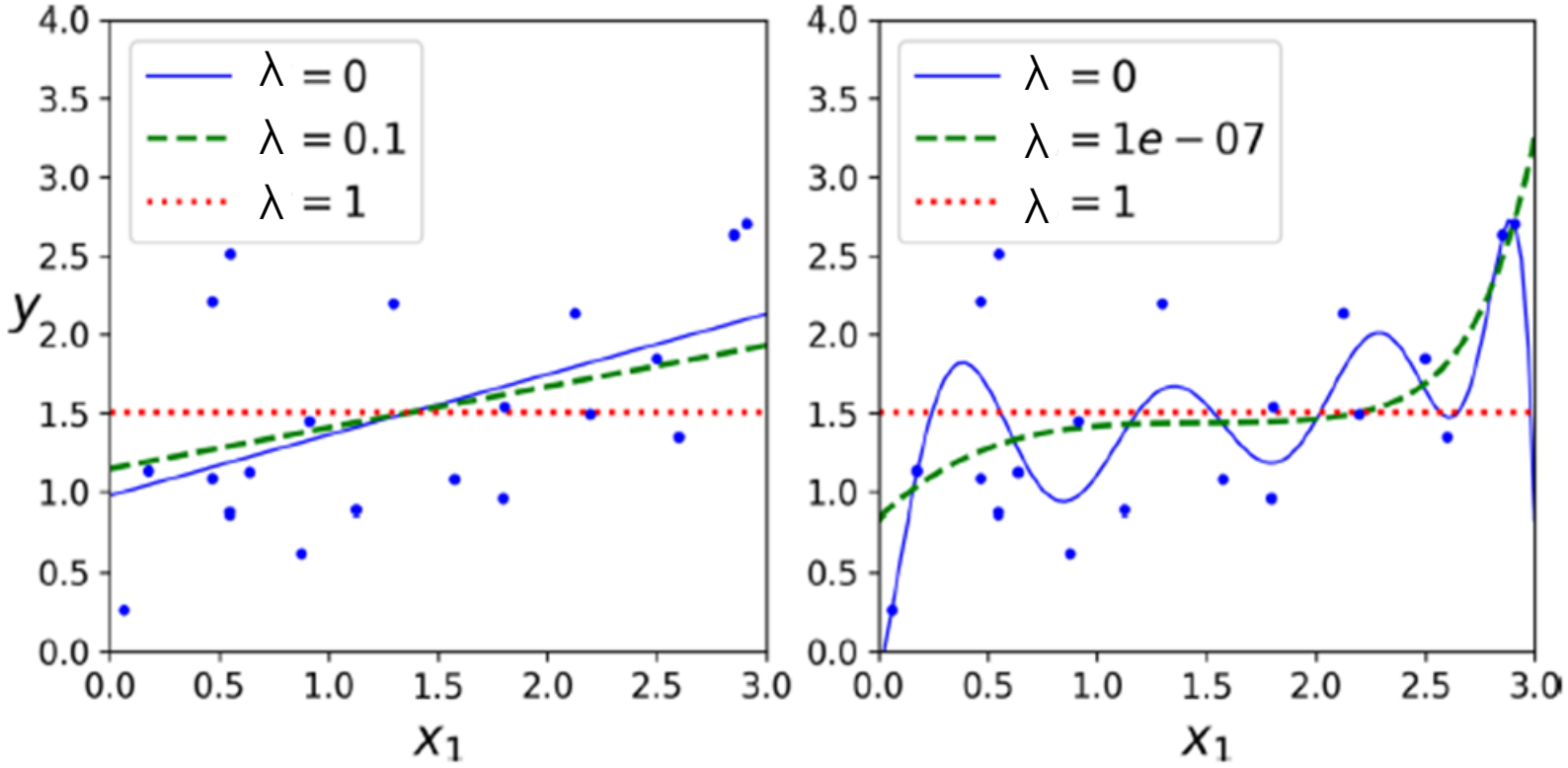


Lasso Regression

- $\mathrm{CF}=\frac{1}{m}\sum_{i=1}^{m}\ (h_{\theta}(x^{(i)})-y^{(i)})^{2}+\lambda\times|\mathrm{slope}|$

$$\mathrm{Slope}=\sum_{j=1}^{n}\theta_{j}$$

Lasso Regression



Elastic-net Regression

$$\mathrm{CF}=\frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)})-y^{(i)}\right)^2+\lambda_1\times(\mathrm{slope})^2+\lambda_2\times|\mathrm{slope}|$$

Tip

- Use Ridge regression when: Most features are useful, You believe many variables have small, real effects. You don’t want to throw any away

- Use Lasso regression when: You suspect many features are irrelevant, Lasso will set some coefficients to exactly zero. **You want a simpler, interpretable model,** fewer variables → easier to explain

- Ridge: shrink everything
- Lasso: shrink + eliminate
- Elastic-net: Combines both

Assignment 1: Predicting Moisture Content

•**Goal:** Predict the moisture content of wood using sensor measurements.
•Traditionally, moisture content has been determined through oven drying and weighting (Destructive approach)

•**Why it matters:**
•Reduces carbon emissions (less oven drying).
•Saves material (non-destructive method).
•Helps maintain optimal calorific value (efficient burning).

Understand the Data
You have a dataset
where:
Columns 1–6: Sensor measurements (features / inputs).
Last column: Moisture content (target / output).

Video references

Linear and logistic regression simple explanation (https://www.youtube.com/watch?v=3bvM3NyMiE0)

Detailed explanation:

- Linear regression

(https://www.youtube.com/watch?v=W46UTQ_JDPk&t=4354s)

- Logistic regression (https://www.youtube.com/watch?v=4u81xU7BIOc)